

Updated to latest. Got 10 features, 58 bugfixes, and 28 other changes.  
[code.claude.com/docs/en/changelog](https://code.claude.com/docs/en/changelog) for details

➤ View, I'm going to share a YouTube link video with you. Once you extract all the information from the YouTube video, we're going to discuss something regarding that information, okay? Claude, I need you to use the extractor skill to extract information from this YouTube video. ,link: <https://youtu.be/Fruw822BMBc?si=PrMK8Tq4X76YFmoJ>

Listed 1 directory

There's no skill literally named extractor — the one that handles YouTube extraction here is agent-reach. Using that.

 Skill(agent-reach)  
Successfully loaded skill

Read 1 file

Using agent-reach, platform YouTube via backend yt-dlp.

Ran 5 shell commands  
1 skill available

The extractor skill just became available — switching to it as you asked.

 Skill(extractor)  
Successfully loaded skill

yt-dlp already failed on this video (its local build is a year old and YouTube is rejecting it).  
Moving to the bundled helper.

Ran 2 shell commands

Transcript pulled (22 KB). Reading it.

Read 1 file

Extracted. Here's everything in it.

What this video is about

"5 PROJECTS That Will Actually Get You HIRED In 2026" — Aishwarya Srinivasan (ex-Google/Microsoft/IBM, ~10 yrs in ML/AI, now running her own AI startup), published 2026-09-10, 18:20 long.

The argument: portfolios full of PDF chatbots, travel planners, and generic mulTs prove you can follow tutorials — not that you can do forward-deployed

The argument: portfolios full of PDF chatbots, travel planners, and generic multi-agent research assistants prove you can follow tutorials — not that you can do forward-deployed engineering (FDE). An FDE walks into a messy customer environment, understands the workflow, connects systems never designed to talk to each other, and makes AI reliable enough for the stakes. She gives five projects that demonstrate that instead.

The filter she sets up first (01:00–02:00)

Per current OpenAI/Google FDE job descriptions — an embedded builder owning discovery, scoping, system design, implementation, debugging, production rollout. So a portfolio must prove four things:

1. You discover the real problem instead of accepting the first feature request.
2. You build across the stack — models, APIs, data, permissions, deployment.
3. You understand production AI — evals, observability, security, human-approval% needing humans.

3. Document intelligence & approval (07:30–09:30) — invoice approval, contract review, insurance claims, clinical prior auth. Using invoices: layout-aware extraction → validate vendor/line items → match against purchase order → apply approval policy → route exceptions to a reviewer. Core principle: separate probabilistic work from deterministic control — model extracts from ugly scans, code verifies totals, tolerances, duplicates, thresholds; human approves low-confidence/high-value exceptions. Don't stop at extraction — value is in the match and the decision (two-way match, or three-way with goods receipt). Inject failure cases: duplicate invoice, changed bank account, missing PO, mismatched quantity, unreadable field. Eval at multiple levels: field accuracy, match accuracy, exception recall, false approvals, latency, review time saved. "98% field extraction can still be dangerous if the missing 2% is the bank account or total."

4. Customer data onboarding / integration pipeline (09:30–11:30) — she calls this one of the most relevant, since a surprising amount of real deployment work is data mapping, API integration, validation, and exceptions nobody mentioned during sales. Customer sends CRM + billing + support data; schemas don't match, IDs inconsistent, three date formats, docs incomplete. Build a small onboarding portal: upload sample/connect mock API → profile data → propose mapping to your canonical schema → explain uncertain mappings → ask for confirmation. Then deterministic validation (types, required fields, referential integrity, duplicates, business rules); valid records proceed, invalid go to an exception

queue with reason + suggested fix. The model proposes mappings, never silently rewrites customer data; mappings stay reviewable and versioned, with schema-drift detection showing exactly which mapping broke. Expose via API, add idempotency so retries don't duplicate, deploy with logs and alerts, document the path to a customer VPC. Include one mid-project change request (new required field) and show the architecture absorbs it without a rewrite.

5. Operations command center with an action loop (11:30–13:30) — support ops, infra reliability, sales pipeline, fraud, or marketplace supply. Answers three questions every morning: what changed, why it matters, what to do next. Compute metrics in SQL/Python, detect anomalies statistically (moving averages, thresholds, change-point detection), and only then let the model explain and propose actions. "The model should narrate the truth, not calculate the truth" — don't hand it 10,000 rows and ask for the weekly SLA rate; pass computed results with provenance and require every claim to point back to a metric. Close the loop: drill into accounts/regions/queues, draft a Jira ticket or internal update, final action behind human approval. Add observability for model calls, tool errors, latency, cost. Bonus: replay a month of data to show how it would have behaved.

How to make any of them read as FDE work (13:30–16:00)

- One-page discovery brief first: customer, users, current workflow, pain point, systems, constraints, success metric. Fictional customer is fine — believable detail is not optional. Bad: "a support team wants an AI agent." Good: "a 20-person support team manually triages 800 tickets/week across three product lines, and priority mistakes are causing SLA breaches."
- Show architecture and tradeoffs — why retrieval vs. workflow graph vs. rules engine vs. queue, and what you deliberately did not automate; how identity, permissions, retries, and failure modes flow through.
- One complete vertical slice beats six pretty screens with mocked backends. Boring stack is fine: Python, FastAPI, Postgres, a queue, one model provider, smallest orchestration layer that fits.
- Deploy it. Logs, a runbook, usable by someone else without you sitting next to them.
- Eval section with real failure analysis: stale source, invalid structured out permission, out-of-policy request — and what the system did about each.
- README answers six questions: who's the customer, what was broken, what did you build, how does it work, how did you evaluate it, what would you change before production.
- 3-minute demo starting with the business problem, showing the workflow and onn the first minute on the architecture diagram.
- Build in public. Share the discovery brief, architecture decisions, eval failures that forced changes. "It's not about 500 likes — it's about the five right people." FDE roles are hard to hire for, and recruiters are actively hunting on LinkedIn.

On AI coding tools (16:00–16:45)

Use Codex or any coding agent to accelerate implementation, review architecture, generate tests, debug. Don't let it make every product and technical decision while you watch — in an interview you have to explain why the workflow has those boundaries, why the evals measure those failures, and what you'd do when requirements change. "If you cannot explain those decisions, the repository is not going to save you."

Her recommended sequence (16:45–18:00)

Build project 1 and project 2 first — together they cover retrieval, permissions, tool use, state, human approvals, and evals. Then pick one domain project matching your target company: financial services/healthcare → document approval (3); integration + deployment strength → data onboarding (4); analytics/ops → command center (5). Two deep projects with a clear customer story are enough. You do not need all five — two done well and documented beat five half-finished repos.

Closing takeaway: an FDE portfolio isn't a collection of AI features, it's evidence you can take an ambiguous customer problem through discovery → design → implementation → evaluation → rollout.

### Caveats

- Transcript is auto-captioned: her name renders as "Ashwatha Srinivas" in-audio (channel is Aishwarya Srinivasan), and a few OCR-ish slips appear — "FDA signal" for FDE signal, "item potency" for idempotency, "20% support team" for 20-person, "20% support team manually triggers" for triages.
- Promotional: she and co-founder Arvind are launching Gen Academy and a masterclass on AI for forward-deployed engineers dated 13 September — which is before this video's stated publish date of Sept 10, 2026, so that date likely refers to a different year or a re-run. Verify in the description before relying on it.
- Everything here is caption-only — I haven't seen slides, code, or on-screen diagrams.
- Claims about OpenAI/Google FDE job descriptions and the hiring market are her characterization, worth checking against live postings.

Ready when you are — what did you want to discuss?

✱ Baked for 1m 41s · done Thursday 1:09 PM

➤ Keep her recommendation aside. What do you think? Which project should I do first? I'm going to do is not something complicated, but something easier to do, so that once I do the project completely, I need to get this type of confidence: I can do this stuff, and I should do this thing. Out of the five, obviously, we need an easier project. The second thing I'm looking for is that it should have meaning. Once I share this with other people in my network, it should feel like it has to have some weight to it. That's what I'm looking for. Please do it. Suggest to me which one I should go for first.

Project 2 — the intake-to-resolution workflow. Not as a warm-up before the "real" ones, but because it's the one that scores highest on both of your criteria at once.

That overlaps with her list, but for a different reason and in a different order: she pairs 1+2 for coverage. I'm saying do 2 alone, first, and don't touch 1 until it's shipped.

Why 2 is the easiest to actually finish

The other four each hide a cost that shows up in week two, after you've already told people you started:

- Project 1 needs chunking, embeddings, a vector store, hybrid search, reranking, and a permission model layered under all of it. Five moving parts that fail in ways that are hard to tell apart — when an answer is wrong, you don't know if retrieval missed, the reranker buried it, or the prompt drifted. That's a confidence-destroying debugging loop for a first project.
- Project 3 dies on document parsing. Layout-aware extraction from ugly scans item wearing a business-workflow costume.

Triage is a workflow every single person in your network recognizes without explanation. You don't have to teach them the problem before you show the solution. And the claim you get to make at the end is concrete and checkable:

■ "I replayed 800 real issues through it. 91% routed correctly. Zero missed sev-1s, because severity escalation never touches the model."

That last clause is the one that reads as engineering judgment rather than tutorial-following. It's a small sentence that says you know where models belong and where they don't.

The specific version I'd build

Bug/issue triage for an open-source developer tool.

The reason to pick this exact domain over IT tickets or HR requests: the data is real and already labeled. Pull closed issues from a large public repo via the GitHub API. Maintainers already applied labels, priorities, and assignments — that's your ground truth, free, and nobody can accuse you of grading your own homework on synthetic data you invented.

That one choice kills the two things that usually stall these projects: no data to build on, and no honest way to evaluate.

Your discovery brief writes itself, and it's specific in the way she asked for:

■ A 6-person maintainer team on a repo receiving ~120 issues/week. Triage is manual, inconsistent between maintainers, and security-relevant reports sometimes sit for days because they arrive looking like ordinary bug reports.

### Build order

1. Discovery brief + eval set first. Pull 300 closed issues, hold out 100. Do this before any model code — it's what stops you from endlessly polishing prompts with no scoreboard.
2. One model call returning a typed object: category, severity, confidence, affected component, suggested owner, next action.
3. Deterministic policy layer in plain Python. Anything matching known security signals escalates regardless of what the model said. This is the part that makes it credible — write it as real code with real tests.
4. Four routes: needs-more-info (auto-comment asking for a repro), auto-labeled and routed, escalated, or held for human review on low confidence.
5. Human-in-the-loop with real state. A reviewer can pause, edit the proposed labels, approve, and resume. Persist state to Postgres so it survives a restart — that's the difference between a demo and a system.
6. Replay the held-out 100. Confusion matrix, missed escalations, false escalations, % needing human review. Then dig into the failures by category and write down what you found.

a state machine in a table and a few functions is cleaner, and choosing the smaller abstraction is the exact senior signal she described.

### What to cut when you're tired

Cut the UI to a bare table with approve/edit buttons. Cut the auto-comment posting to a dry-run that logs what it would post. Cut multi-repo support. Do not cut the eval replay or the failure analysis — that section is 80% of why anyone will take the project seriously, and it's the cheapest part to build.

### Realistic scope

Two focused weeks. One if you're going hard. It's the only one of the five I'd expect you to finish without the ending getting rushed — and finishing cleanly is the entire point of the first one.

Once it's shipped and you've written the failure analysis, Project 1 gets much easier to attempt, because you'll already have the eval discipline and the human-approval pattern built. That's the real reason to sequence it this way.

✱ Crunched for 50s · done Thursday 1:12 PM

› Okay, let's build project 2 then. I need you to go into a deep preset stage and launch multiple agents, like 5 to 6 agents, each working on a different thing.

First, segregate:

- what the particular project is
- what different areas the project covers
- which tool can be used for each particular task
- which are the easier tools to use
- which are the harder tools to use
- why to use a particular tool
- the tradeoffs between using that and the other tools
- for each particular task, a free tool that can be used if it's possible
- how the integration works

Ask me if you need any more questions before starting the research

Good — before I launch the agents, four things that would change what they research. Everything else I'll decide myself.

 User answered Claude's questions:

Research backend and state machine  
Research evaluation harness  
Research deployment and observability  
Research UI and packaging

All six are running in parallel. Here's how I split the project so nothing overlaps and nothing falls through:

The project, stated plainly

AI-assisted issue triage, evaluated against real maintainer decisions. Closed issues from a large public repo come in; one LLM call proposes a typed decision; a deterministic policy layer overrides it on security/severity signals; the request routes to one of four paths; a human reviews and approves in a UI with persisted state; and a held-out set of real issues gets replayed to produce honest numbers.

The reason this works as a first project: the ground truth is free and real. Maintainers already labeled those issues. You're not grading your own homework.

The six areas

1 — Data & ground truth. GitHub API access paths, which specific repos have label taxonomies clean enough to use, how to map messy real labels onto a small schema, temporal vs random splitting to avoid leakage, corpus storage and caching so you're not re-

hitting the API all day, and how many issues the numbers actually need to mean something.

2 — The LLM extraction layer. Every free-tier provider with real current limits, and the arithmetic on whether each can actually push ~1000 classifications through without hitting a daily cap. Structured-output approaches compared on reliability and debuggability. Retry-and-repair when the model returns garbage, calibrated confidence, and response caching so eval reruns are free.

3 — Backend, state machine, policy layer. Framework and DB choices with TypeScript given a fair hearing rather than dismissed. The pause/edit/approve/resume state machine — and I explicitly asked this agent to stress-test my earlier "hand-roll it, skip LangGraph" claim rather than just agree with me. Plus idempotency, the audit-trail schema, and how to express policy rules so you can record which rule fired.

4 — Evaluation & failure analysis. The area carrying most of the credibility. Metric design with missed escalations as the dangerous one, calibration, and — the part I pushed hardest on — ablations against a dumb baseline. If TF-IDF + logistic regression gets within two points of your LLM, you need to know that before a hiring manager asks. Also statistical honesty: confidence intervals at  $n=100$  vs  $n=1000$ , and when a difference between runs is just noise.

5 — Deployment & observability. Free hosting with a genuine public URL, free Postgres, and one risk I flagged loudly: several free databases expire or get deleted after N days of inactivity, which would quietly kill your portfolio link months after you share it. The agent is specifically hunting for that, plus cold-start mitigations for when someone opens your link in three months.

6 — Review UI & packaging. Minimal table-with-approve-buttons options, and one architectural question that decides your whole deployment shape: Streamlit/Gradio run as their own server (two services to deploy) versus FastAPI + HTMX (one). Plus read-only public demo mode so a live link isn't world-writable, README structure, diagram tooling, and a concrete build-in-public post sequence.

Every agent has the same required output shape you asked for: what the area covers, candidate tools, easier vs harder explicitly ranked, why each, tradeoffs, a free option per task with real limits, how integration works, and one clear recommendation. All were told to verify against live sources rather than recall — free tiers change constantly and that's where stale answers do the most damage.

I'll synthesize all six into a single build plan when they land, and flag where any two of them contradict each other — that usually marks the decisions that actually matter.

✧ Worked for 3m 10s · done Thursday 1:18 PM

› Message from @a85ee654b04b2b05f (ctrl+o to expand)

Area 1 (data & ground truth) is back. Five still running. The decision-relevant parts:

Repo choice shifted. I'd assumed Kubernetes as the obvious pick; the agent found its priority taxonomy is great but it has no first-class security label — k8s routes security disclosures through a private process entirely. That's a problem, because "never miss a sev-1" is the headline claim of your project, and it needs a real security signal to test against.

Its primary recommendation is rust-lang/rust instead: P-low / P-medium / P-high / P-critical is a clean ordinal severity scale that maps straight onto your policy layer's escalation logic. elastic/kibana is the other strong candidate (impact:critical/high/low plus a Team:Security label). Both still need their security labeling verified by eye before you commit — the agent couldn't confirm it from search alone and said so rather than guessing, which I'd rather have.

One methodology point that matters more than it sounds: temporal split, not random. Sort by close date, train on the older issues, hold out the most recent. Random splitting leaks, because near-duplicate issues cluster in time and share the labeling conventionches the real scenario — triaging issues nobody has seen yet.

A number worth knowing now: you need roughly 150–300 holdout issues before the hing, 500–800 total. And report per-class support counts — "94% accuracy" on a class with 4 examples isn't a claim worth making. Flagging that yourself in the than being asked about it in an interview.

A nice side effect: fetching 1000 issues is ~10 API calls and finishes in under a minute, well inside the free 5,000/hr authenticated limit. Cache the raw pull to JSONL once and every later iteration — prompt tweaks, mapping changes, eval reruns — runs fully offl nothing and never becomes a bottleneck.

I'll hold the rest until the other five land, then reconcile them — particularly agent 3 on whether my "skip LangGraph" call survives scrutiny, and agent 5 on which free database won't quietly delete itself in three months.

✧ Baked for 13s · done Thursday 1:18 PM

 Agent "Research data layer tooling" finished · 2m 25s

That's just the completion notification for the data-layer agent I already coveive agents still running; I'll report when the next lands.

› Message from @af363f6b62f444b54 (ctrl+o to expand)

Area 4 (evaluation) is back — this is the one carrying most of the project's weight, and it came back with a clear verdict.

Skip the eval platforms. All of them: promptfoo, DeepEval, Braintrust, Weave, Inspect AI, Langfuse's eval features. Not because they're bad, but because they're built for a problem you don't have — semantic judges, RAG faithfulness, agent trajectories, red-teaming. Your task is closed-label classification with real ground truth, which is exactly what sklearn.metrics was built for. The recommendation is pytest + scikit-learn as the metric core, plus MLflow in local file-store mode (no server, no account, data never leaves your machine) purely for free run history and comparison.

Two timing details worth knowing: OpenAI is acquiring promptfoo (announced March 2026), and OpenAI's hosted Evals platform goes read-only Oct 31 2026 and shuts down Nov 30 — bad things to build a new dependency on right now. Listing these as "evaluated and consciously skipped, here's the one-line reason" is itself an interview answer.

The part that matters most — the ablation. Run the same golden set four ways: model-only, rules-only, combined, and trivial baselines (majority class + TF-IDF/logistic regression). One CLI flag, same pipeline, different predictor. The sentence you're aiming to be able to say:

■ "The rules layer alone catches X% of true escalations on its own. The LLM's marginal contribution is Y points of missed-escalation reduction over rules-only, and Z points over the TF-IDF baseline."

And the agent's honest addendum, which I agree with: if the LLM doesn't clearly beat rules + TF-IDF, that's a legitimate finding, not a failed project. Writing that up truthfully is a stronger signal than a suspiciously clean win.

Your headline number is not accuracy. It's missed-escalation rate — false negatives on severity. That's the failure with asymmetric cost, and it's what an interviewer will probe.

Two rigor moves that are cheap and rare:

Statistical honesty. At ~85% accuracy, the 95% CI is roughly  $\pm 7$  points at  $n=100$ ,  $\pm 4$  at  $n=300$ ,  $\pm 2.2$  at  $n=1000$ . So at your realistic holdout size, a 2–3 point bump between runs is probably noise. Use McNemar's test on the paired items before claiming a prompt change helped.

The label-noise ceiling. Hand-relabel 30–50 issues yourself and measure how often you disagree with the maintainer. If you disagree on 12 of 50, your reported accuracy is

bounded by a ~24% noise ceiling and no model can exceed it. Publishing that number is one of the strongest credibility signals available to you, and almost nobody does it.

Four agents still out: LLM layer, backend/state machine, deployment, and UI/pac

Area 2 (the LLM layer) is back. It did the arithmetic I asked for, and the throughput math kills several obvious options.

Free tiers, measured against a 1,000-issue replay:

| Provider              |                                                                                             | Verdict |
|-----------------------|---------------------------------------------------------------------------------------------|---------|
| Gemini 2.0 Flash-Lite | 1,500/day, 30/min → ~33–60 min, room for retries. Winner.                                   |         |
| Mistral free numbers  | ~17 min, best raw throughput — but they've stopped publishing fixed                         |         |
| Groq (llama-3.3-70b)  | 1,000/day cap = one full run uses your entire daily budget, zero retry headroom. Fine ≤500. |         |
| Cerebras              | 5 req/min → 3.3+ hours. No.                                                                 |         |
| OpenRouter free       | 50/day → 10–20 days unless you deposit \$10. No.                                            |         |
| GitHub Models         | Retired 2026-07-30. Dead.                                                                   |         |

|             |                                                                            |
|-------------|----------------------------------------------------------------------------|
|             |                                                                            |
| Together AI | No standing free tier since mid-2025 — just a \$5 signup credit. Not free. |
|             |                                                                            |

Recommendation: Gemini 2.0 Flash-Lite via the native SDK — real constrained decoding through responseSchema (not prompt-and-pray), no credit card, and the only free option that exposes logprobs. Caveat to re-check: Google cut free quotas 50–80% in Dec 2025 and could again.

Structured output: Instructor + Pydantic. Not raw parsing, not BAML, not Outlines. It's the easiest option that's still honest about failures — it raises a catchable `ValidationError`, which is exactly what you want to log. LangChain's `with_structured_output` got marked down specifically for burying errors under abstraction layers.

A genuinely clever suggestion I hadn't considered: run the same batch through two providers — Gemini (real schema constraint, structurally can't emit bad JSON) and Groq or a small Ollama model (weaker guarantees, will occasionally emit garbage). That gives you a real measured malformed-output rate to write up. The weaker provider isn't a problem, it's your failure-analysis data. Instructor makes it a config change rather than a rewrite.

On the confidence field — important. Self-reported LLM confidence ("rate yourserated; it clusters at round numbers and near the ceiling. Don't present it as agent wants the raw corpus in JSONL/SQLite, this one sketched a Postgres issues table. Easy fix — raw corpus stays flat-file, Postgres holds live app state.

Three still out: backend/state machine, deployment, UI/packaging.

✧ Worked for 18s · done Thursday 1:19 PM

🔵 Agent "Research LLM extraction layer" finished · 2m 48s

🔵 Agent "Research backend and state machine" finished · 2m 39s

The backend agent finished, but its report didn't actually arrive in my context — only the completion notice did. Requesting it rather than guessing at its contents.

› Message from @ac6073e0e999f74a3 (ctrl+o to expand)

It arrived. Area 3 (backend & state machine) — and my "skip LangGraph" call survived the stress test, but with a sharper edge than I gave it.

The verdict, honestly argued. The agent agreed the hand-rolled state machine is correct, but did the work of saying when it wouldn't be. The four routes are a fixed lookup from (category, severity, confidence) — not a content-dependent graph.

ALLOWED\_TRANSITIONS: dict[str, set[str]] plus a status column plus an immutable decision\_log table replaces the entire state-machine library in about 15 lines, and "resume" needs no mechanism at all: the row's current status is the resume point. One source of truth, no second persistence system to reconcile.

The trigger it named for when LangGraph would earn its place: if the single classification call grows into genuine agentic work — the model requesting more GitHub context, checking for duplicate issues, looping with a critique step. That threshold isn't crossed here. Naming the trigger is what makes the rejection read as judgment rather than ignorance.

It also caught something I'd have missed: once more than one person can review, two reviewers can race on the same row. Cheap fix — an optimistic-concurrency version/updated\_at check on the update. Doesn't need a framework, but does need to exist.

Database warnings, corroborated independently (this agent found them without being asked — the deployment agent is checking the same ground, so I'll see whether they agree):

- Render free Postgres: 30-day hard expiry, then deleted after a 14-day grace period. This would silently destroy your project a month after you share the link.
- Supabase free: pauses after 7 days idle, manual resume required. Bad for a "public URL that's always reachable."
- Fly.io: no meaningful free tier for new accounts since Oct 2024.
- Neon free: autosuspends on idle but never deletes — 0.5GB, 100 CU-hours/month

Stack recommendation: FastAPI + SQLAlchemy 2.x/Alembic + Neon. FastAPI over Litning benchmarks — you're never QPS-bound at triage scale, and at 11pm whensomething breaks, ecosystem depth beats throughput. TypeScript got a fair hearing and lost on migration tooling maturity, not on taste.

No task queue. Not Celery, not arq, not even procrastinate. The batch replay is a finite, on-demand, single-process job — asyncio.gather with a Semaphore to respect your Gemini rate limits. And a persistent Celery worker is exactly what a sleeping free-tier instance is worst at hosting.

Policy layer: plain Python functions, each returning (decision, rule\_id, reason) which wins. One function = one rule = one pytest test = one row in your decisionlog. A YAML DSL would buy "non-engineers can edit rules," which you don't need.

The data model it sketched is clean, and decision\_log is the piece carrying your audit story — "why did issue #42 escalate" becomes one SQL query.

Two agents left: deployment/observability and UI/packaging. Then I'll reconcile everything, including whether the deployment agent's free-tier findings match this one's.

✧ Worked for 22s · done Thursday 1:20 PM

› Message from @ab403707fdb01a8e6 (ctrl+o to expand)

Area 5 (deployment) is back, and it independently corroborates the backend agent on every database finding — two agents, separate searches, same conclusions. That's the strongest signal in this whole batch.

The confirmed landmines:

- Render free Postgres: deleted at ~44 days (30-day expiry + 14-day grace). The agent called this the single most likely way your project silently dies months after you share it — and named the exact beginner mistake that causes it: co-locating the free app and free DB on the same platform.
- Supabase: pauses at 7 days idle, needs a manual dashboard click. Not self-healing. A hiring manager opening your link gets a 500.
- Neon: scale-to-zero after 5 min, but never deletes. ~1s cold start, data persists indefinitely, no card. This is non-negotiable.
- Fly.io free tier dead for new accounts. Deta shut down in 2024. Koyeb closed to new signups after Mistral acquired them (Feb 2026). Cloud Run now requires a billing account as of Feb 3 2026 — still \$0, but a card on file.

Recommended stack: Render free web service + Neon on a separate provider + strue (50k units/mo) + Sentry free (5k events/mo) + GitHub Actions (unlimited on public repos) + UptimeRobot keep-alive.

The decoupling is the point: Render's 15-minute sleep (recently cut from 30) used database is data loss. Keeping them on separate providers means the two risks never compound.

The mitigation I'd treat as mandatory — DEMO\_MODE. Your public URL has three problems nobody thinks about until they bite:

1. A visitor clicking "approve" corrupts your seeded demo data.
2. A bot hitting a triage endpoint burns your LLM key on unbounded calls.

3. Someone opens the link cold in three months and stares at a 30–60s blank load.

Fix: gate every mutating endpoint behind a DEMO\_MODE env var that no-ops writes with a friendly message, serve pre-computed replay results rather than live LLM calls to the public, seed the DB so the first page view is populated rather than empty, and put a fast static fallback page in front. Roughly an afternoon of work that decides whether your link is impressive or embarrassing on the day it actually matters.

Two small things worth catching now: set SQLAlchemy pool\_size=2-5 and use Neon's pgbouncer endpoint — Render's 512MB/0.1 CPU instance will fall over on default pool settings. And mock the LLM client in CI, or every push burns real API quota.

🔵 Agent "Research UI and packaging" finished · 2m 23s

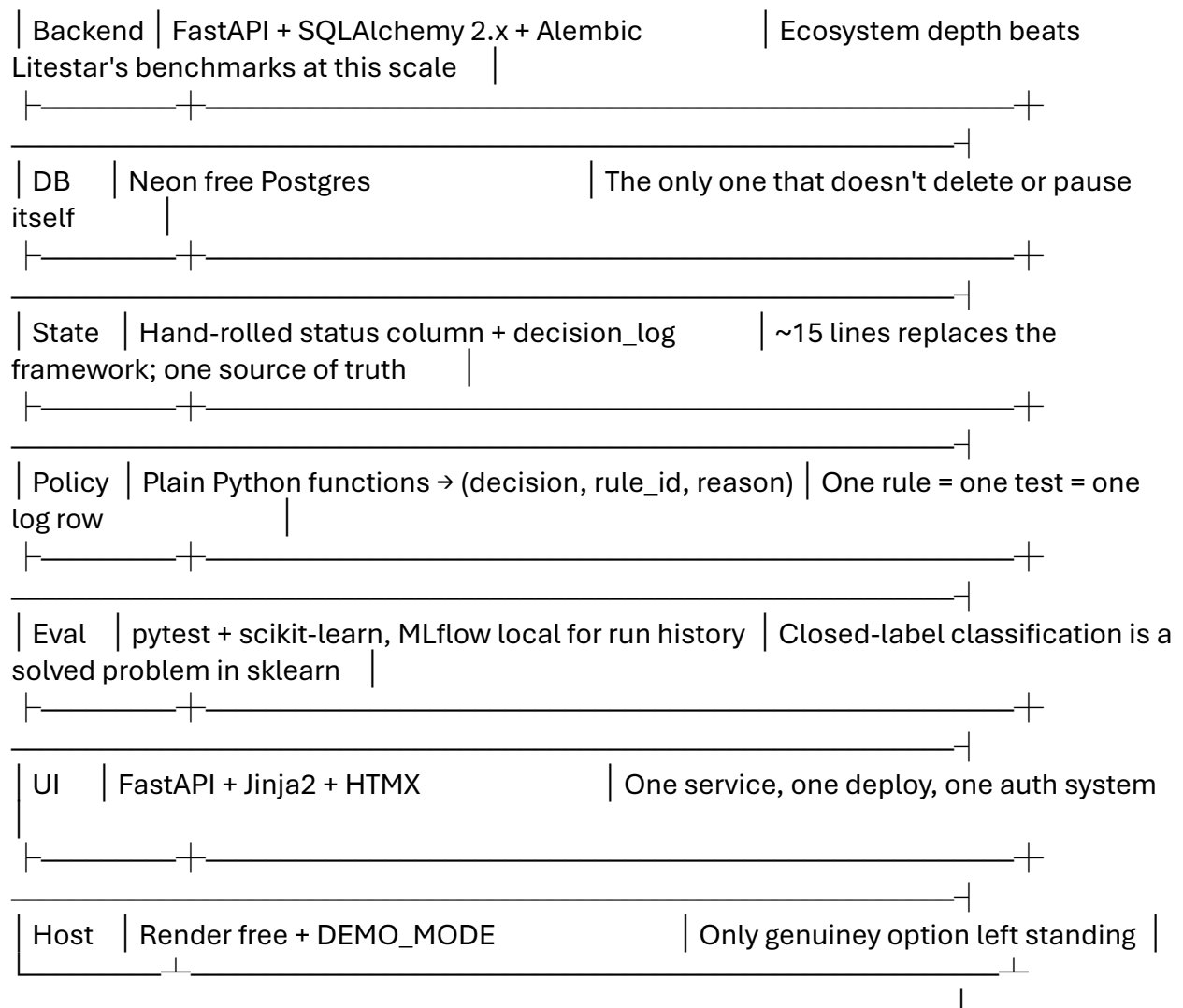
The UI/packaging agent finished but its report hasn't landed in my context — same as happened with the backend one. Requesting it rather than guessing.  
Resuming agent a3d80ce

› Message from @a3d80ce1a953fa6bc (ctrl+o to expand)

All six are in. Here's the reconciled plan.

The stack — locked

| Layer  | Choice                                          | Why it won                                                    |
|--------|-------------------------------------------------|---------------------------------------------------------------|
| Data   | GitHub REST, authenticated, JSONL cache on disk | ~10 calls for 1000 issues; everything downstream runs offline |
| Model  | Gemini 2.0 Flash-Lite, native SDK               | Only free tier with headroom for 1000 + retries (1,500/day)   |
| Schema | Instructor + Pydantic                           | Easiest option that still raises catchable errors you can log |



Where independent agents agreed

Three separate agents, searching separately, converged on the same database findings — that's the highest-confidence material here:

- Render free Postgres deletes itself at ~44 days. Never point the app at it.
- Supabase pauses at 7 days idle and needs a manual dashboard click. Not self-healing.
- Neon scale-to-zero, never deletes. The pick.
- Fly.io free tier dead, Deta shut down, Koyeb closed to new signups, Cloud Run now needs a card.

The UI agent and the backend agent also arrived at FastAPI independently, from different directions — one from deployment topology, one from ecosystem maturity.

Where they conflicted, and my calls

Corpus storage. Data agent wanted JSONL/SQLite; backend agent sketched a Postgrright about different things. Raw immutable pull → JSONL on disk (your cache,el there. Kibana has Team:Security plus impact:critical/high/low. Spend 20 minutes eyeballing both label pages before writing any code — this choice determines whether your headline metric is measurable at all.

## Build order

The one non-obvious sequencing decision: run your dumb baselines on day 3, before the LLM exists.

|      |                                                                           |  |
|------|---------------------------------------------------------------------------|--|
|      |                                                                           |  |
| Days | Work                                                                      |  |
| 1    | Pick repo (verify labels by eye), write ingest script, cache to JSONL     |  |
| 2    | Label mapping → ground truth, temporal split, report per-class counts     |  |
| 3    | Baselines: majority class + TF-IDF/LogReg. Get the number to beat         |  |
| 4    | LLM classify: Instructor + Pydantic + response cache + failure logging    |  |
| 5    | Policy layer, rules as functions, decision_log writes in same transaction |  |
| 6    | Eval harness, confusion matrix, four-way ablation table                   |  |
|      |                                                                           |  |

|     |                                                                                |  |
|-----|--------------------------------------------------------------------------------|--|
| 7   | Hand-label 50 issues yourself → label-noise ceiling                            |  |
| 8–9 | FastAPI + HTMX review table, expand-row, edit, approve, rule badge             |  |
| 10  | Deploy: Render + Neon + DEMO_MODE + seeded data + fallback page                |  |
| 11  | structlog, Sentry, GitHub Actions CI with mocked LLM, failure-analysis writeup |  |
| 12  | README (six questions), Mermaid diagram, discovery brief in docs/              |  |
| 13  | 3-minute demo: QuickTime + iMovie                                              |  |

Doing baselines first means the ablation is structural, not retrofitted — and if the LLM turns out not to beat TF-IDF by much, you find out on day 4 instead of day 13.

Re-verify before you build

Free tiers move fast enough that "verified today" has a shelf life of weeks: Gemini's free quotas (Google cut them 50–80% in Dec 2025), Render's sleep timeout and Neon's CU-hour cap, the chosen repo's live label list, and htmx — pin an exact 2.0.x version, not latest.

What actually makes this credible

Four things, in order of how rare they are:

1. The label-noise ceiling — hand-label 50, report your disagreement rate, state that no model beats it.
2. The four-way ablation — model-only, rules-only, combined, dumb baseline.
3. Raw counts beside every percentage — "94.2% (312/331)", never a bare percentage.
4. 3–5 real misrouted issues linked, with what went wrong and why.

Everything else is table stakes. These four are what separate it from a tutorial.

Want me to write this up as a build-plan page you can keep open and check off over the two weeks, or start on day 1 — the repo label verification and the ingest script?

✧ Worked for 1m 1s · done Thursday 1:21 PM

› Message from @a3d80ce1a953fa6bc (ctrl+o to expand)

That's the re-send of the UI/packaging report arriving after I'd already received and folded it into the synthesis — nothing new in it.

So all six areas are covered and the plan above stands. The question still open is which way you want to go:

- Write it up as a build-plan page you can keep open and check off across the two weeks, or
- Start on day 1 — verify the label taxonomy on rust-lang/rust and elastic/kibana, pick one, and write the ingest script.

If you're ready to build, I'd go straight to day 1. The repo decision is a 20-minute look at two label pages and it gates everything else.

✧ Crunched for 7s · done Thursday 1:22 PM

 Agent "Research UI and packaging" finished · 4m 43s

Duplicate notification from the re-send — nothing new. All six agents are done and the plan stands; waiting on your call between writing it up as a checkable build-plan page or starting day 1 with the repo label verification.

✧ Worked for 8m 58s · done Thursday 1:22 PM

✧ recap: You're building an AI issue-triage portfolio project, and six research agents finished; the full stack and two-week plan are decided. Next: choose whether I write the plan up as a checkable page or start day 1 by verifying label taxonomies on rust-lang/rust and elastic/kibana. (disable recaps in /config)

› <https://www.youtube.com/watch?v=FB-MLPhL9Ms>, see if any of this information is useful for the evaluation we will be doing

Read 1 file, ran 1 shell command

Extracted. It's "The maturity phases of running evals" by Phil Hetzel (Solutions Engineering lead at Braintrust), AI Engineer conference, 18:33, published May 2026.

Worth knowing up front: he works for an eval vendor, and your eval agent already recommended skipping Braintrust for this project. The theory half is vendor-neutral and genuinely useful; the tooling half is a booth pitch.

Also, most of the talk targets a different eval shape than yours. He's addressing agent evals — subjective outputs, LLM-as-judge, multi-step traces, tool calls, external system state. You have discrete labels and real ground truth, which puts you in an easier and more rigorous position. But three things transfer directly, and two of them should change the plan.

Change 1 — your day-7 noise audit gets an upgrade

His "just getting started" advice: when a human annotator gives a thumbs up/down, force them to write a justification. His reasoning is that the domain knowledge is stuck in the annotator's head, and the justification is how you extract it.

Then the next stage: run those justifications through an LLM to derive the actu

Applied to you: when you hand-label those 50 issues and disagree with the maintainer, don't just record that you disagreed — write one line on why. Then cluster those fifty why-lines into failure categories. That turns a flat noise percentage ("24% disagreement") into something far better: "here are the four systematic ways maintainer labels are wrong, and here's which of my model's errors are actually the ground truth being wrong." That's a substantially stronger README section for maybe 30 extra minutes of work.

Change 2 — your review UI is a flywheel, and I missed that

His central diagram: capture production traces → find what went wrong → bring those examples back into the offline eval set → re-run → improve.

Your review UI already produces exactly this and I didn't connect it. Every time a reviewer edits or overrides a routing decision, that's a new human-verified label — often better than the maintainer's original. So:

- decision\_log rows where a human overrode the system become golden-set v2 candidates
- Your eval set grows from the act of using the product
- You get a real answer to "how would this improve in production?" in your README's final section — not hand-waving, but a working mechanism

That's a genuine addition to the plan. Small to build (you're already logging the data), strong to talk about.

Validation — three of your existing choices got independent support

"Evals are not unit tests." Start from known failure modes; don't try to exhaustively cover everything, because it's infinite and you'll spend all your time writing tests instead of shipping. That's the argument for your failure-analysis approach over chasing coverage.

"Don't think about evals as running tests — think about evals as re-running production." That's precisely what your temporal split plus held-out replay of real closed issues does. Good confirmation of the split decision.

Deterministic scoring where the failure mode is objective. He explicitly says you don't need an LLM judge when code can catch it — too many tool calls, too many tokens. Same principle as your deterministic policy layer.

What doesn't apply — and why that's good news

His level-3 and level-4 material — trace-level eval, mocking external systems, faking system state at eval time, timestamped vector DB queries — is all irrelevant to you. Your system makes one stateless LLM call per issue. No tools, no CRUD, no state to reconstruct.

And the whole LLM-as-judge thread, including the closing Q&A about deterministically evaluating things you get to sidestep entirely. He says you should "eval the eval" because judge outputs need their own ground truth. You already have ground truth — real maintainer labels — so you need no judge at all and no judge-calibration problem.

That's actually a point worth making out loud in an interview: "I didn't need LLM-as-judge because I chose a problem with real ground truth. That's a dataset design decision, not a tooling one." It's the kind of thing that reads as having thought a level deeper than the tutorial.

Want me to fold both changes into the plan — justifications in the noise audit, and the review-UI-as-flywheel loop — and then start on day 1?

✧ Crunched for 49s · done Thursday 1:53 PM

Are we building the second one to confirm again?  
8 skills available

Yes — project 2, the intake-to-resolution workflow. That's what everything so far has been for.

Concretely, the version we scoped: AI-assisted issue triage built on real GitHub issues, where maintainers' own labels on closed issues give you free ground truth.

The flow, unchanged since you picked it:

1. Ingest closed issues from a public repo
2. One LLM call returns a typed decision (category, severity, confidence, component, owner, next action)
3. A deterministic policy layer in plain Python overrides and escalates on security/severity signals
4. Route to one of four paths — needs-more-info, auto-label-and-route, escalate, hold-for-human-review
5. A human reviews, edits, approves in a minimal UI; state persists so it can pause and resume
6. Replay the held-out real issues to get honest numbers, with missed escalations as the headline

The six agents researched tooling for that and only that, the stack is locked, and there's a 14-day build order ready. The one open item is which repo — rust-lang/rust vs elastic/kibana — which is a 20-minute look at two label pages.

Two pending additions from the Braintrust talk I'd fold in before starting: written justifications in the day-7 noise audit, and treating reviewer overrides as new golden-set data.

Ready to start day 1 whenever you are.

✧ Baked for 9s · done Thursday 6:31 PM

➤ /Users/Checkout/Downloads/FDE\_Portfolio\_Handout\_Aishwarya\_Srinivasan.pdf forgot dropping this read it

Read 1 file, ran 1 shell command

Read it — 9 pages, and it's a companion to the same video. Page 4 is entirely Project 2.

First, a correction to something I said after the transcript: I flagged the Sept 13 workshop date as conflicting with the Sept 10 publish date. It doesn't — the 13th is three days after the 10th, both 2026. The handout confirms it. That workshop has already passed (today is the 17th).

What's genuinely new for your build

Her exact schema for the decision object — and it differs from ours in ways worth deciding on deliberately:

```
class Triage(BaseModel):  
    category: str
```

urgency: Literal["p1", "p2", "p3"]  
confidence: float  
policy\_basis: str  
owner: str  
next\_action: Literal["ask\_info", "draft\_reply", "route", "escalate"]

Three observations:

2. next\_action maps exactly onto our four routes. ask\_info / draft\_reply / route / escalate versus our needs-more-info / auto-label-and-route / escalate / hold-for-human-review. Near-identical, independently arrived at. Good sign.

3. policy\_basis as a model output — I'd deviate here, deliberately. Her schema has the model state which policy applies. Our design puts that in decision\_log.rule\_id + reason, written by deterministic code. I think ours is stronger and consistent with her own rule on the same page ("deterministic checks in code, judgment in the model") — if the model names the policy basis, the policy is back inside the model. But I'd have the model emit a short free-text reasoning field for the review UI to display, which is the useful half of what policy\_basis was doing.

Her evaluation table adds one metric we'd named differently and one that doesn't apply:

| Her metric                                   | Ours                                 |  |
|----------------------------------------------|--------------------------------------|--|
| Routing accuracy                             | same                                 |  |
| False escalations                            | same                                 |  |
| Missed escalations — "the dangerous failure" | same, and independently our headline |  |
| Tool call success                            | N/A — you have no tools              |  |
| Human intervention rate                      | our "% needing human review"         |  |

What it confirms

The LangGraph guidance is on the page verbatim: "If the flow is fixed and simple, ordinary application code is completely fine. Choosing the smaller abstraction is a senior engineering signal." Your backend agent stress-tested that and reached the same place. That decision is now supported from three directions.

Page 8's packaging advice — six README questions, 3-minute demo opening with the business problem, build-in-public, "ship the smallest complete solution, then measure where it fails" — matches what the UI/packaging agent recommended almost point for point.

One resource worth actually using

Page 9 points to her own post, "AI Evals Explained for Builders" ([aishwaryasrinivasan.substack.com/p/ai-evals-explained-for-builders](https://aishwaryasrinivasan.substack.com/p/ai-evals-explained-for-builders)), described as the framework behind every "how you evaluate it" section in the handout — heuristic, statistical, and model-based evals. Her instruction: "Use the evals post when you write the evaluation section."

Given evals are the credibility core of your project, that's worth pulling before day 6. Want me to fetch and extract it the same way?

Net effect on the plan

Two small changes: three urgency levels instead of four (better statistics), and add a reasoning field to the model output for the review UI while keeping policy basis in deterministic code. Everything else stands.

Still pending from the Braintrust talk: justifications in the day-7 noise audit, and reviewer overrides feeding the golden set.

Say the word and I'll fold all four in and start day 1.

✧ Sautéed for 50s · done Thursday 6:34 PM

✧ recap: You're building project 2, an AI issue-triage system on real GitHub issues, to prove FDE skills. Research is done and the stack is locked. Next: verify label taxonomies on rust-lang/rust vs elastic/kibana, pick one, and write the ingest script. (disable recaps in /config)

➤ Hey, I have read the handout and watched the video multiple times regarding the project. I still don't have a perfect idea of what we are trying to build and what we are going to build.

Can you just give me a basic idea of how this stuff actually works? Is there something I need to concentrate on? Explain the whole problem to me clearly.

What I got here after watching the video was that, usually, in each and every company, there will be issues from the customer side. The issue the customer has might be gotten from a support ticket, or they might be emailing the team, or it's something else. Our workflow should be designed in a way where the agent should understand the particular issue and divide it into four different categories. She said there are four categories, and it should keep the deterministic part and the model part separate.

I know these are just some pieces, and this doesn't stitch together exactly. I need you to explain to me in detail, regarding this project, what the YouTuber is trying to tell me to build, and how I should explain it to me with examples.

Your mental model is about 80% right. There's one piece mis-wired, and fixing it makes the rest snap together.

The one correction

You said the agent divides the issue into four categories. It doesn't. The four things are four actions the system takes next — not categories of issue.

Category is a separate field (bug, feature request, docs, question...). The fousult:

1. Ask for missing info — the report is unusable as-is
2. Draft a reply / auto-label and route — routine, handle it
3. Route to the correct team — it's clear, send it to whoever owns it
4. Escalate to a human — this is serious or the system isn't sure

So the system reads an issue, works out what it is, and then decides what to do about it. That second part is the whole point.

Why this isn't a chatbot

This is the thing to hold onto. A chatbot's output is text for a human to read. This system's output is a decision the system acts on.

Nobody chats with it. An issue arrives, and at the end a row in a database says: this is a p1 security bug in the auth module, it goes to the security team, and a human must approve before it moves.

That's why she picked this project to demonstrate FDE work. Anyone can make an LLM produce a paragraph. Making it produce a decision that a company would actually act on is a different job, because now being wrong has consequences.

The problem, in the real world

Every company has a front door where messy requests arrive — support tickets, etc reports. A human reads each one, works out what it means, checks how urgent it is, and sends it to the right place.

That work is boring, constant, and inconsistent. Two people triage the same ticket Friday, something urgent gets mislabeled as routine and sits for four days.

Your system does that first pass automatically, and a human confirms it.

What actually happens to one issue

Five stages. Each one is a distinct piece of code.

Stage 1 — Intake. An issue arrives: a title and a body of text. That's it. Mess incoherent.

Stage 2 — The model reads it. One LLM call. It returns a typed object — not prose:

```
{  
  "category": "bug",  
  "urgency": "p2",  
  "confidence": 0.81,  
  "component": "auth",  
  "owner": "platform-team",  
  "next_action": "route"  
}
```

This is the only place the AI is involved. It reads unstructured text and turns it into structured fields. That's its entire job.

Stage 3 — The policy layer checks it. Plain Python. No AI. It looks at both the issue text and what the model said, and applies your company's hard rules. It can override the model.

Stage 4 — Route. Based on the final decision, the issue goes down one of the four paths.

Stage 5 — A human reviews. They see the proposal, can edit it, and approve or reject. Nothing is final until they do.

Four worked examples

Here's the same system handling four different issues.

---

Issue A: "App crashes" — body is empty.

Model: category bug, urgency p3, confidence 0.2, next\_action ask\_info.

Nothing is actionable here. No version, no steps, no error. → Ask for missing imminent asking for reproduction steps. No human time spent.

---

Issue B: "Typo in the README — 'recieve' should be 'receive'."

Model: category docs, urgency p3, confidence 0.96, component docs, next\_action route.

Obvious and low-stakes. → Auto-label and route to the docs team. High confidence, no human needed.

---

Issue C — this is the important one. "Password reset link still works after being used. I used my reset link, then clicked it again an hour later and it logged me straight in."

The model reads this and returns: category bug, urgency p3, confidence 0.74.

The model got this wrong. It read a calm, politely-worded bug report and scored it as routine. But this is an authentication bypass — a real security hole.

Now the policy layer runs:

```
def rule_security_signal(issue, classification):
    SECURITY_TERMS = ["auth bypass", "password reset", "session",
                      "token", "privilege", "leak", ...]
    if any(term in issue.body.lower() for term in SECURITY_TERMS):
        return Decision(
            route="escalate",
            urgency="p1",
            rule_id="SEC-001",
            reason="Security-sensitive term in body; escalation is mandatory."
        )
```

The rule fires. The model said p3; the system escalates anyway. → Escalate to a human, immediately, flagged p1.

This single example is the heart of the whole project. Here's why:

If  
you had trusted the model alone, a real security hole would have sat in the nistic rule

caught it — not because the rule is smart, but because it doesn't depend on the model having a good day. It's a keyword check. It fires the same

Her line from the handout: "If a security ticket contains a known severity-one signal, escalation must not depend on whether the model happens to reason correctly that day."

That's the sentence the whole project exists to demonstrate.

---

Issue D: "Memory usage grows over time in long-running processes" — a long, detailed, technically ambiguous report.

Model: category bug, urgency p2, confidence 0.42.

No rule fires, but confidence is low. → Hold for human review. A person looks at it and decides.

Why split the model from the code

You now have the answer from Issue C, but let me state it directly.

The model is good at judgment and bad at guarantees. It can read a rambling paragraph and work out that it's a billing complaint. But ask it the same question tomorrow and you might get a slightly different answer. It has no obligation to be consistent.

Code is bad at judgment and perfect at guarantees. if "password reset" in body cannot have an off day. It will fire every time, forever.

So you give each one the job it's actually good at:

| The model does               | The code does                |  |
|------------------------------|------------------------------|--|
| Read messy text              | Enforce security rules       |  |
| Infer category and component | Enforce severity floors      |  |
| Suggest an owner             | Decide the final route       |  |
| Give a confidence score      | Log which rule fired and why |  |

And critically — the code runs last and can overrule the model. Not the other way

Not just a checkbox. Three real properties:

1. Pause — the issue sits at `held_for_review` and nothing else happens
2. Edit — the reviewer can change the proposed urgency or owner before approving
3. Resume from the same state — they can close the laptop, come back tomorrow, and the issue is exactly where they left it

That third one is why your database has a status column. The row's status is the workflow's position. There's no separate engine holding state in memory that vanishes on restart.

And every transition writes a row to `decision_log`: what changed, who changed it, which rule fired, why. So months later you can answer "why did issue #4821 get escalated?" with one query. That's the audit trail.

How your GitHub version maps onto this

She describes support tickets. You're using GitHub issues. Same shape, one big advantage:

| Her example                  | Yours                                 |
|------------------------------|---------------------------------------|
| Support tickets arriving     | Closed GitHub issues from a real repo |
| A human triages them         | Maintainers already did — years ago   |
| You'd need to grade yourself | Their labels are your answer key      |

That's the whole reason we picked this. You're not inventing test data and hoping it's realistic. You're replaying a thousand real issues that real maintainers already triaged, and checking whether your system agrees with them.

When your system says "p1, security, escalate" and the maintainer labeled it P-critical — that's a match, scored automatically. Do that a few hundred times and you have a real accuracy number.

What to concentrate on

Three things, in order:

1. The system's output is a decision, not text. If you can hold that one idea, itself.
2. The model proposes; the code disposes. The model never gets the final word on anything that matters. Issue C is your entire project in one example — if you can tell that story out loud in an interview, you've demonstrated the thing FDE hiring actually screens for.
3. Missed escalations are the number that counts. Not accuracy. A system that routes 95% of issues correctly but misses one security report is worse than one that's right 88% of the time and never misses a critical. Asymmetric cost — that's why it's your headline metric.

Everything else — the framework, the database, the hosting — is plumbing. Real work, but plumbing.

Does that land? If any stage still feels fuzzy, tell me which one and I'll go deeper on just that piece before we start building.

✧ Baked for 56s · done Friday 12:24 PM

› Yeah, it's completely clear for me right now. Let's move on to something else, which is nothing but GitHub issues or what we want in GitHub issues. How can we find this? Which particular repo should we use? How can we decide which one to use? I have no clue which report I should select for this particular task, so I need your complete help here.

Let me actually look at the data rather than theorize. Checking the candidate repos' live label sets.

Ran 2 shell commands

Labels existing isn't the same as labels being used. Now counting how many closed issues actually carry them.

Ran 2 shell commands

Kibana has by far the most labeled data. But large repos often have hidden noise — let me check what those issues actually are.

Ran 3 shell commands

I pulled real numbers from GitHub for the three strongest candidates rather than going on what the research agents claimed. Two of their claims turned out to be wrong, which I'll come back to.

What a repo has to give you

Your system makes decisions, and you're going to grade those decisions against what the maintainers actually did. So the repo is your answer key, and it needs:

1. A severity scale maintainers actually use — critical/high/medium/low or similar. This is what tells you whether your system should have escalated.
2. Enough critical issues. Missed escalations are your headline number, and you can only measure them on issues that should have been escalated. With 10 critical issues in your test set, one miss swings the result by 10%.
3. Owner/team labels, so you can check the "route to the right team" part.
4. Type labels (bug, feature, docs, question) for category.
5. Recent and consistent labeling. Labels drift over the years, so we only use issues closed since 2023.
6. Readable issue text. A hiring manager should be able to follow your demo.

What the data showed

All counts are closed issues since Jan 2023.

|                       | rust-lang/rust                                                   | elastic/kibana                  | kubernetes/kubernetes |
|-----------------------|------------------------------------------------------------------|---------------------------------|-----------------------|
| Closed issues         | 14,854                                                           | 44,032                          | 7,353                 |
| Have a severity label | ~989                                                             | ~7,949                          | ~982                  |
| Critical issues       | 112                                                              | 465                             | 116                   |
| Severity labels       | P-critical/high/medium/low<br>priority/critical-urgent...backlog | impact:critical/high/medium/low |                       |
| Owner labels          | T-* teams                                                        | 117 Team:* labels               | sig/*                 |
| Readability           | Hard: compiler internals<br>hard, lots of CI noise               | Medium: product bugs            | Medium-               |



The number that decides it is the critical count. You'll hold out about a quarter of your data for the final test:

- Rust: 112 critical → about 28 in the test set. Too thin to say anything confident about missed escalations.
- Kubernetes: 116, and most of the samples I looked at were CI job failures ("e2e jobs failing since..."), not user-reported problems.
- Kibana: 465 critical. After removing noise (below) that's about 370, so roughly 90 in the test set. Enough to report a missed-escalation rate with a real confidence interval.

Recommendation: elastic/kibana

It has 8× the labeled data of the other two and enough critical issues to measure. Its labels also line up with the handout's three urgency levels:

| Kibana label                        | Your field           |  |
|-------------------------------------|----------------------|--|
| impact:critical                     | p1 → should escalate |  |
| impact:high                         | p2                   |  |
| impact:medium, impact:low           | p3                   |  |
| bug / enhancement / docs / Question | category             |  |
| Team:*                              | owner                |  |

Sample critical titles, so you can see what the data looks like:

- "Entities from an existing Kibana space are visible in a newly created space" (a data leak across tenants, genuinely p1)
- "Dashboard fails to open when controls have title: null"
- "SentinelOne Response Actions Failing with 500 Error"

Anyone can follow these.

Three problems in Kibana, and how to handle them

1. Bot-generated test failures. About 21% of critical issues (96 of 465) are titled "Failing test: ..." and were filed by CI bots. They aren't human requests and nothing needs triaging. Drop any issue whose title starts with "Failing test".

2. The "security" trap. This is the most interesting finding. Kibana ships a product called Elastic Security, so "Security" appears in hundreds of titles as a product name: "[Security Solution] Error on navigating to rules page". A naive policy rule of if "security" in text: escalate would fire on every one of them and bury the review queue in false escalations.

So your security rule has to look for vulnerability language: auth bypass, privilege escalation, XSS, visible across spaces/tenants, leaks, exposes credentials. It can't just look for the word "security". That's a real design problem you'll hit, measure and explain, and it's a good failure-analysis story to have: "My first rule fired on the product name. Here's the false-escalation rate before and after I fixed it."

3. Label drift. Some Team:\* labels are marked - DEPRECATED and others are duplicates (Team:actionable-obs vs Team:ActionableObservability). You'll write a small mapping that merges these into about 10–15 clean owner groups. Also exclude impact:needs-assessment, which means maintainers hadn't judged the issue yet, so it isn't an answer.

Two corrections to earlier research

- I overstated how much you need a security label. Your "should this escalate?" answer key is simply impact:critical. Security keyword rules are one tool in your policy layer for catching those issues, and you grade the rules on whether they catch critical issues. So the repo doesn't need a dedicated security label, which also takes the pressure off the choice.
- The data agent said Kubernetes had no security label. It has area/security (59 uses since 2023). It also treated Kibana's Team:Security as a vulnerability signal, but that's the team that owns the Security product. Neither changes the recommendation.

What we'd pull

Closed issues from elastic/kibana since 2023 that have an impact:critical/high/g "Failing test" titles and bot authors. That's roughly 7,500 candidates, and we'd take a sample of about 1,000, deliberately keeping all the critical ones so p1 isn't starved. We split by close date: older issues to develop against, the newest ~25% held back for the final test.

On licensing: Kibana's code is under Elastic's own license, but that doesn't govern issue text, which is public on GitHub. The safe approach for a portfolio is to keep the raw data on your machine and show only a handful of example issues in the public demo, never the full dataset.

If you're happy with Kibana, the next step is the ingest script: fetch those issues, cache them locally as JSONL, and print the per-class counts so you can see the real distribution before we write anything else. Want me to write it?

✱ Baked for 3m 17s · done Friday 12:37 PM

✂ recap: You're building an AI issue-triage system for your FDE portfolio, and we just picked elastic/kibana as the data source because it has the most labeled critical issues. Next step: say yes and I'll write the ingest script. (disable recaps in /config)

› Okay, I have wondered before. Going forward, I just wanted to confirm: is it possible for us to shift this particular chat to a different folder? I don't want to build the chat in this section. It's just brainstorming for me. The projects are built in a different folder in my account. If yes, how can we do that?

Yes. Let me check a couple of things so I give you the right steps rather than guessing.

Ran 1 shell command

A chat session can't be moved between folders, but I can either write a CLAUDE.md with all our decisions into the new project folder so a fresh session there picks up full context, or use /add-dir to edit files there while staying in this chat. I don't see a project folder yet besides portfolio/—where should it go?

🗨 User answered Claude's questions:

- Where should the triage project live? → New: projects/issue-triage
- Which handoff approach? → B: /add-dir, stay in this chat

Going with projects/issue-triage and staying in this chat. /add-dir needs the f I'll create it first.

Ran 1 shell command

The folder exists and is empty: /Users/Checkout/Documents/projects/issue-triage.

The next step has to come from you, because /add-dir is a slash command and I ce this in the prompt:

/add-dir /Users/Checkout/Documents/projects/issue-triage

After that I can create and edit files in issue-triage while we keep going in this chat. Once it's added, I'd start with two files:

1. CLAUDE.md records every decision we've made: the project, the stack, Kibana and its label mapping, the "Failing test" and "security" pitfalls, and the build order. We're staying in this chat, so it isn't strictly needed now. But any future session opened in that folder picks it up automatically, and it's the raw material for your README.
2. The ingest script fetches Kibana's labeled closed issues since 2023, caches them locally as JSONL, and prints how many issues fall into each urgency level. That shows you the real distribution before anything else is built.

✳ Brewed for 39s · done Friday 12:58 PM

➤ which is a better name issue triage or the handout name what do you think

Go with issue-triage. The handout name has two problems.

It says "I followed the video." "Intake-to-resolution workflow" is Aishwarya's label for the project category, and plenty of people have watched that video. A hiring manager who has seen it will read the repo name as "project 2 from the handout" before opening anything warns against exactly this: portfolios that prove you can follow a tutorial. Your repo name should describe what you built.

It overclaims. Your system doesn't resolve anything. It triages: it reads the issue, decides what should happen, and hands it to the right person. The resolving is still done by humans. A name that promises more than the system does is a small credibility cost you called resolution if it doesn't resolve?" is an easy interview question to walk into.

"issue-triage" says what the system does, is accurate, and is the kind of search term a stranger would use.

If you want something more specific, kibana-issue-triage names the data source, rather than synthetic. The weakness is that it ties the name to one repo if you ever try another. I'd keep plain issue-triage for the folder and decide the public GitHub repo name when you publish, since you can rename it then with no cost.

The folder is already issue-triage, so nothing to change. Run the /add-dir command whenever you're ready and I'll start on CLAUDE.md and the ingest script.

✳ Sautéed for 11s · done Friday 1:00 PM

✳ recap: We're building an AI issue-triage portfolio project on real Kibana GitHub issues, and the issue-triage folder is created. Next, run /add-dir